KNN and the Bayes Boundary

This notebook opens Chapter 5. We meet a simple classifier called **KNN**, short for "k nearest neighbors", and see it score perfectly against the data on which it was "trained". Then we revisit the **Bayes classifier**, the best possible classifier theoretically, but usually impossible to implement. We see that it can't score perfectly. Both facts are true at once. Sorting out why leads to the central question of Chapter 5: how do we measure how well a model will do on new data?

By the end, you'll be able to

1. Describe how KNN classifies a new point by letting its nearest training points vote.
2. Read a decision-boundary plot, and explain how the number of neighbors controls how flexible the boundary is.
3. Explain what the Bayes boundary is, how we can draw it for simulated data, and why even it misclassifies some points.
4. Explain how a model can beat the best possible classifier on its own training data, and why that's a warning sign rather than a success.
5. Explain why a model's score on the data it was fit to can't tell us how it will do on new data.
6. Explain bias and variance, which describe how a method's boundaries behave on average over many training sets, and use them to explain why error on new data falls and then rises as the model gets more flexible.
7. Describe the validation set approach and its weaknesses.
8. Describe k-fold cross-validation, and explain why it agrees with itself far better than a single validation split does.
9. Use cross-validation to choose K from the training data alone, then refit the chosen model on all of it.

> **How to read the code in this notebook.** There are two kinds of code cells.
>
> - **Code worth reading.** Cells that use standard libraries, such as pandas (`pd.`) and scikit-learn, do everyday tasks you'll do yourself: fitting a KNN model, computing an error rate, building a small table. Each is introduced with **Code worth reading** in bold. Read these line by line.
> - **Demo cells.** Cells that import from `ch_5_demos_helpers` draw a plot or build an interactive demo. That file was written for this

> course, and its plotting code isn't the point. You never need to open it. Ignore this kind of code and focus on how the resulting plots and demos fit into the lesson.

In [1]:
```
import numpy as np
import pandas as pd
```

1 · KNN: how one prediction is made

Suppose we have training points whose true class labels we know, and a new point x_0 whose class we want to predict. **K-nearest neighbors (KNN)** finds the training points closest to x_0 and lets them vote. The number of neighbors that vote is K, a setting we choose ourselves. We write $\mathcal{N}_0$ for the K training points closest to x_0.

> **A note on symbols.** ISL uses the letter K for three different things: the number of classes (equation 4.15), the number of neighbors in KNN, and, later in Chapter 5, the number of folds in cross-validation (written with a lowercase k). In this notebook, K always means the number of neighbors. We only have two classes, so we write them as class 1 and class 2.

Here is the whole algorithm:

```
To classify a new point x0:
    1. Compute the distance from x0 to every training point.
    2. Take the K training points with the smallest
distances.
    3. Count how many of those K are in each class.
    4. Predict the class with the most votes.
```

KNN is based on distances, so features measured on very different scales can distort them. You'll see this in the lab.

Demo A: How KNN votes

The plot shows 16 training points with two features, X_1 and X_2: 8 circles (class 1) and 8 triangles (class 2). The star is the point we're classifying, x_0. Faint lines show its distance to every training point. The K nearest are highlighted, inside a dashed circle that reaches the Kth nearest one.

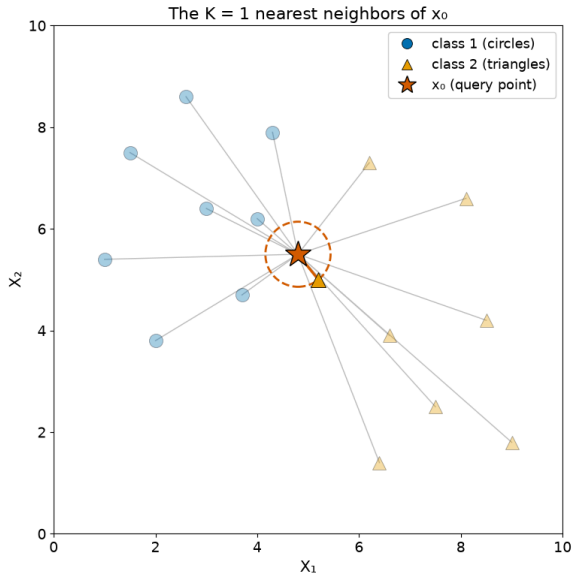
Try each preset, or move x_0 with the sliders. Then pick **Near the boundary** and move K from 1 to 5. (K takes only odd values, so two classes can't tie.)

In [2]:
```
from ch_5_demos_helpers import knn_vote_demo

knn_vote_demo(size_percent=80)
```

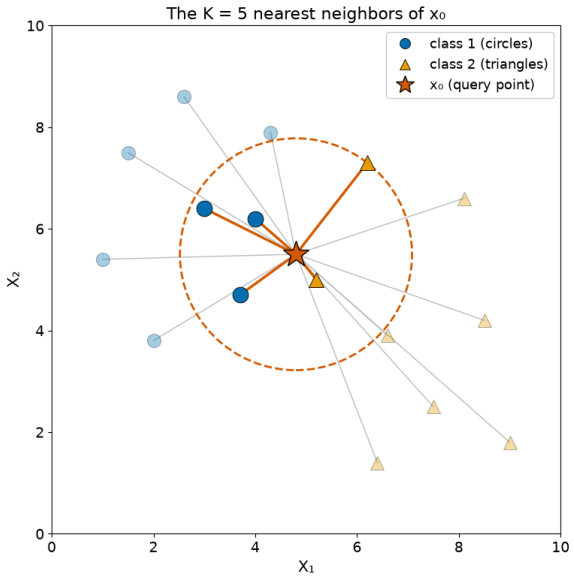
PDF version: the demo is shown at the settings below. Run the notebook to explore it.

Near the boundary, K = 1



0 circles, 1 triangle → **predict triangle** (0/1 = 0% circle)

Near the boundary, K = 5



3 circles, 2 triangles → **predict circle** (3/5 = 60% circle)

The vote is a probability estimate. The readout's fraction, such as "3/5 = 60% circle," is KNN's estimate of the probability that x_0 is in class 1:

$$\Pr(Y = 1 \mid X = x_0) \approx \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = 1)$$

In words: the fraction of the K nearest neighbors that are circles.

The I is an **indicator function**: it turns a yes-or-no question into a number. $I(y_i = 1)$ asks "is training point i a 1 (a circle)" and results in 1 for yes and 0 for no. Adding up those 1s and 0s over the K neighbors in $\mathcal{N}_0$ (read "N-naught", or "the neighborhood of x_0") counts the circles, and dividing by K turns the count into a fraction.

Ask: Set the selector to "Near the boundary". At $K = 1$ this point is a triangle; at $K = 5$ it's a circle. Which prediction do you trust more, and why?

▼ Answer

We should usually trust $K = 5$ more than $K = 1$. At $K = 1$ the prediction rests on a single training point, and that one point may have landed where it did by chance. At $K = 5$, five points vote, so no single unusual point decides the

answer. Very large K has its own problem, which Section 2 shows. Later in this notebook, we discuss methods for finding a good value for K.

2 · Decision boundaries, flexibility, and training error

In the real world, we almost never know the underlying function that generated the relationships between the features, X, and the dependent variable, Y. Some parts of this notebook require this knowledge (for teaching purposes), so we're going to simulate a data set:

- 200 simulated training points,
- 100 circles and 100 triangles (these are our two classes: the "ground truth").
- Section 3 explains how they were made.

Code worth reading. `simulate_data` returns the data; the next lines put the training points in a small pandas table and show its first rows. `X[:, 0]` is the first column of `X` (the feature X_1), and `X[:, 1]` is the second (X_2).

In [3]:
```
from ch_5_demos_helpers import simulate_data

sim = simulate_data()                # You don't need to understand this lin
X, y = sim.X, sim.y                  # the 200 training points and their cla

train = pd.DataFrame({'X1': X[:, 0], 'X2': X[:, 1], 'Y': y})
train.head()
```

Out[3]:

	X1	X2	Y
0	1.186036	0.509190	1
1	-0.638818	-0.700683	1
2	1.417814	0.408937	1
3	0.565277	-1.206761	1
4	1.396006	0.525189	1

Reading a decision-boundary plot. Imagine running KNN's vote at every location, (x_1, x_2) on the plot and shading each location by the class KNN would predict there.

- The **decision boundary** is where the shading switches from one class to the other (see the demo below).
- The **training error** is the share of the *training* points (the data the model was fit to) that the model misclassifies.

- **Flexibility.** With small K, each prediction depends on only a few nearby points, so the boundary can bend around individual points: the model is **flexible**.
- With large K, many points vote everywhere, so the boundary is smooth: the model is **rigid** (less flexible). Small K is the flexible end.

Demo B: The K slider

Slide K from the flexible end (left) to the rigid end (right), and watch the boundary and the training error. **Shade by vote fraction** replaces the two flat colors with KNN's estimated probability of class 1 at each location (Section 1's fraction).

- Leave the two buttons off for now. Just mode the K slider.

Ask: Which K will have the lowest training error? Why?

▼ Answer
$K = 1$, with a training error of exactly 0%. Each training point is its own nearest neighbor, at distance 0. So when $K = 1$ predicts a training point, it just looks up that point's own label, and it's always right.

Ask: What will happen to the decision boundary, as K decreases? As K increases?

▼ Answer
As K decreases, the model becomes more flexible, because $\mathcal{N}$ ("neighborhood") shrinks. Fewer "voting" neighbors means the boundary can wiggle more dramatically.

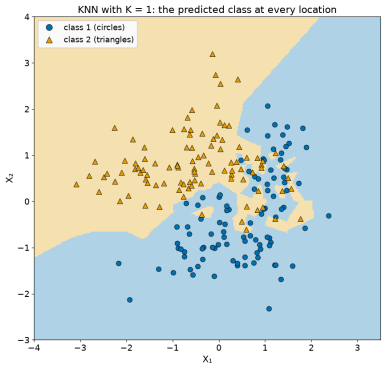
As K increases, the size of $\mathcal{N}$ grows larger. There are more "voting" neighbors, so local variations get cancelled out and the boundary is wiggles less (is smoother).

In [4]:
```
from ch_5_demos_helpers import knn_boundary_demo

demo_b = knn_boundary_demo(sim, K=15, size_percent=80)
```

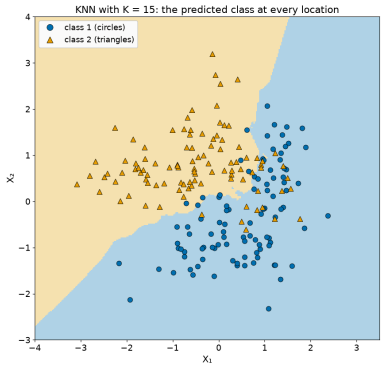
PDF version: the demo is shown at the settings below. Run the notebook to explore it.

K = 1 (most flexible)



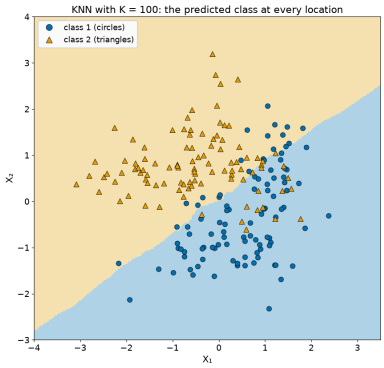
Training error: **0.0%**

K = 15



Training error: **12.0%**

K = 100 (least flexible)



Training error: **18.0%**

Code worth reading. Fitting KNN yourself takes two lines with scikit-learn. `model.score(X, y)` returns the accuracy, the share of points classified correctly, so the training error is 1 minus that. The result matches the demo's readout at $K = 1$.

In [5]:
```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=1)
model.fit(X, y)
print(f'Training error with K = 1: {1 - model.score(X, y):.1%}') # Training
```

```
Training error with K = 1: 0.0%
```

3 · The Bayes boundary: the best possible classifier

Because these data are simulated, we can compute the true probability of each class for any point on the plot using equation 4.15. The computer does that for a fine grid of points covering the plot, then draws the boundary where the two classes are tied at 50/50. If you do this using the *exact* version of Bayes Theorem (equation 4.15), the boundary is called the **Bayes boundary**.

With two classes, equation 4.15 reads

$$\Pr(Y = 1 \mid X = x) = \frac{\pi_1 f_1(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)}$$

In words: the true probability of class 1 at x is class 1's share of the population, π_1, times how densely class 1's points are spread near x, $f_1(x)$, divided by that same product added up over both classes. Here both classes are equally

common, so $\pi_1 = \pi_2 = 0.5$. The **Bayes classifier** predicts class 1 wherever this probability is above 50%, and class 2 elsewhere.

▼ How the data were generated

To make each point, the code picks a class using the priors π_1 and π_2, then draws a point from that class's density, $f_1(x)$ or $f_2(x)$. Here each class's density is a mix of ten small, round, bell-shaped bumps, scattered around a different center for each class. These are the same two ingredients that appear in equation 4.15, which is why the formula that draws the boundary matches the recipe that made the data.

▼ How the boundary is drawn

The Bayes boundary is where the two classes are tied, so start from equation 4.15 written for each class. Both fractions have the same denominator:

$$\Pr(Y = 1 \mid X = x) = \frac{\pi_1 f_1(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)},$$

$$\Pr(Y = 2 \mid X = x) = \frac{\pi_2 f_2(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)}$$

On the boundary the two probabilities are equal:

$$\frac{\pi_1 f_1(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)} = \frac{\pi_2 f_2(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)}$$

The denominator is the same positive number on both sides, so multiplying both sides by it leaves

$$\pi_1 f_1(x) = \pi_2 f_2(x), \qquad \text{or equivalently} \qquad \pi_1 f_1(x) - \pi_2 f_2(x) = 0.$$

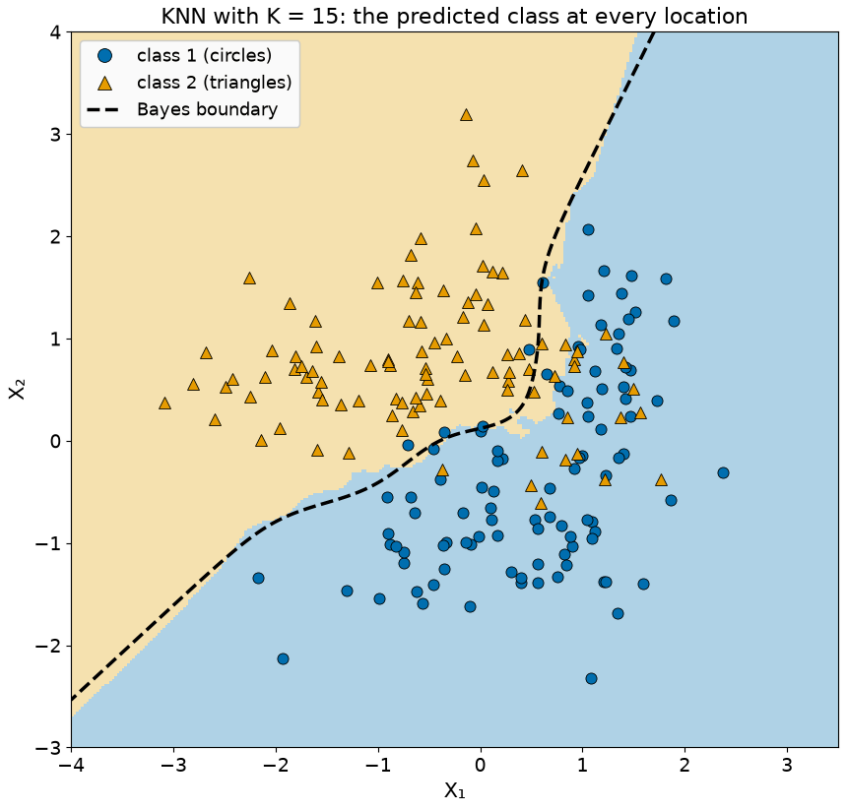
In words: on the boundary, class 1's share times its density equals class 2's share times its density. Where the difference is positive, class 1 is more likely; where it's negative, class 2 is. The computer evaluates $\pi_1 f_1(x) - \pi_2 f_2(x)$ on a fine grid and draws the contour where it equals zero.

Turn on **Show Bayes boundary** in Demo B. The cell below shows the same demo again, so you don't have to scroll back.

In [6]:
```
demo_b
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

K = 15, Bayes boundary on



Training error: **12.0%**
Bayes error rate (on new data): **10.5%**
The Bayes boundary misclassifies **25** of the 200 training points.

The claim. On *new* data, no classifier has a lower expected error rate than the Bayes classifier. Here's why, at one point.

Fix one location x (a whole row of feature values), and suppose the true probabilities there are 70% class 1 and 30% class 2. Any classifier, however it was built, makes one prediction at x. Predict class 1 and you're right 70% of the time; predict class 2 and you're right 30% of the time. Nothing can be right more than 70% of the time at this x, and the Bayes classifier picks exactly that. It makes the best possible choice at every location, so averaged over new data, nothing beats it.

Its error rate is the **Bayes error rate**: at each location, 1 minus the larger of the two probabilities, averaged over the data. The classes overlap at many locations, so the error can't be zero in those regions: any prediction there is sometimes wrong. This is **irreducible error**. Here the overlap comes from the way the bumps overlap. In real data, it comes from things that affect the outcome but aren't among the features, and from variation that no feature could capture,

such as measurement error or plain chance. So "irreducible" means irreducible with these features: adding informative features can lower the Bayes error rate, but nothing guarantees it would reach zero.

Ask: The readout says the Bayes boundary misclassifies some of the 200 training points. Why can't even the best possible classifier get them all right?

▼ Answer
Because the classes overlap. Near the boundary, both classes really occur, and the Bayes classifier predicts whichever is more likely there. The training points on the "wrong" side of the boundary are the less likely class, which still turns up some of the time. No rule can predict those correctly without getting more of the likely class wrong. That's irreducible error.

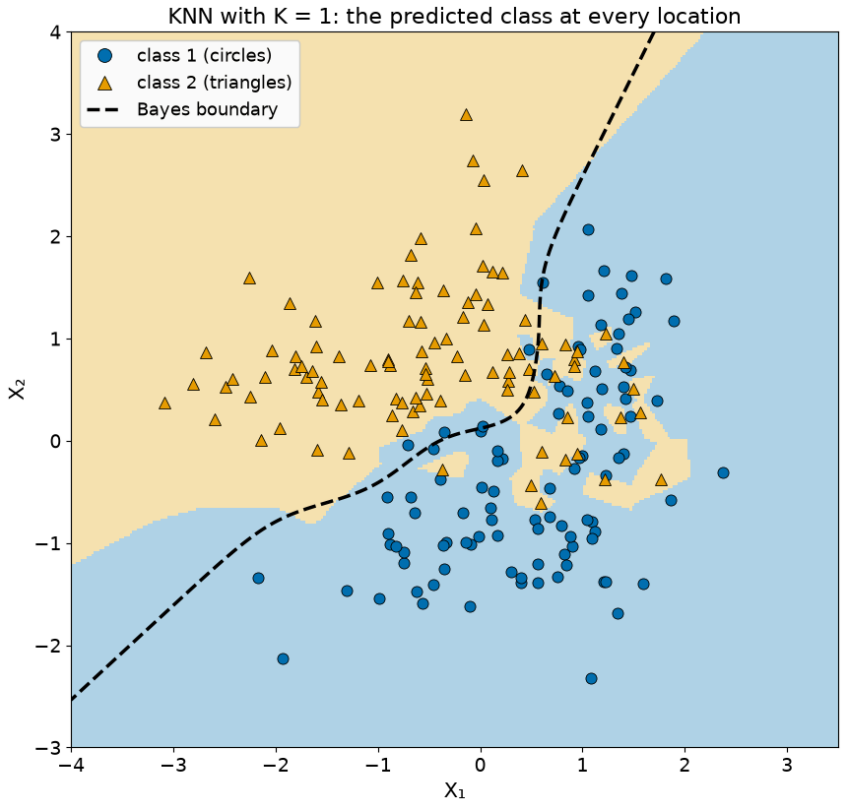
4 · How can $K = 1$ beat the best possible classifier?

Keep the Bayes boundary on and set $K = 1$ in Demo B (shown again below). KNN's training error is 0%, below the Bayes error rate. The Bayes boundary misclassifies about two dozen of the training points, and $K = 1$ misclassifies none of them.

In [7]:
```
demo_b
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

K = 1, Bayes boundary on



Training error: **0.0%**
Bayes error rate (on new data): **10.5%**
The Bayes boundary misclassifies **25** of the 200 training points.

Ask: The Bayes classifier is the best possible, yet $K = 1$ beats it on the training data. How can both be true?

▼ Answer

A trained model like $K = 1$ is a fixed rule, and the argument in Section 3 applies to any *new* point. The training points aren't new: the model has already seen their labels. So $K = 1$ scoring 0% on its training data doesn't contradict anything. It means the model memorized the training points, including the ones that landed on the "wrong" side of the bayes boundary by chance.

That's why a training error below the Bayes error rate is a warning sign. No rule can do that well on new data, so the extra accuracy must come from memorizing chance quirks of these particular points: noise.

5 · Why not just use Bayes?

We never know the true priors, π_j and densities $f_j(x)$ with real data. So every classifier is a way of approximating the Bayes classifier, and each one makes an assumption to get around what it doesn't know.

Approach	Methods	The assumption that fills the gap
Estimate π and f, then use Bayes' theorem	QDA, naive Bayes	f is Gaussian (QDA), or the features are independent within each class (naive Bayes)
Estimate $\Pr(Y = 1 \mid X = x)$ directly	Logistic regression, KNN	The log-odds are linear (logistic regression), or nearby points have similar labels (KNN)

Ask: Which of these methods have we already met? What would you need to know to skip the approximation entirely?

▼ Answer

We've met naive Bayes and logistic regression in Chapter 4, and KNN in this notebook. To skip the approximation, you'd need the true priors π_1, π_2 and the true densities $f_1(x)$, $f_2(x)$. That only happens when you generate the data yourself, as in this simulation.

Demo C2 (optional): What each method assumes, drawn

Every method on Demo C's table fills the gap with an assumption, and you can see each assumption in the shape of the boundary it draws. Logistic regression can only draw a straight line. Naive Bayes and QDA each describe a class as a single smooth blob, so they draw one smooth curve. KNN assumes only that nearby points have similar labels, so it can follow whatever shape the data has.

Code worth reading. The loop fits four models to the same 200 training points, then scores each one on 10,000 new points from the same recipe that the models never saw. That score is the **test error**, which Section 6 looks at closely. `C=np.inf` turns off scikit-learn's default penalty, so `LogisticRegression` fits the plain logistic regression from Chapter 4.

In [8]:
```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

X_test, y_test = sim.X_test, sim.y_test     # 10,000 new points from the sam

models = {'Logistic regression': LogisticRegression(C=np.inf),
          'Gaussian naive Bayes': GaussianNB(),
          'QDA': QuadraticDiscriminantAnalysis(),
          'KNN, K = 15': KNeighborsClassifier(n_neighbors=15)}
```

```
rows = []
for name, model in models.items():
    model.fit(X, y)
    rows.append({'model': name, 'test error': 1 - model.score(X_test, y_test
pd.DataFrame(rows).set_index('model').round(3)
```

Out[8]:

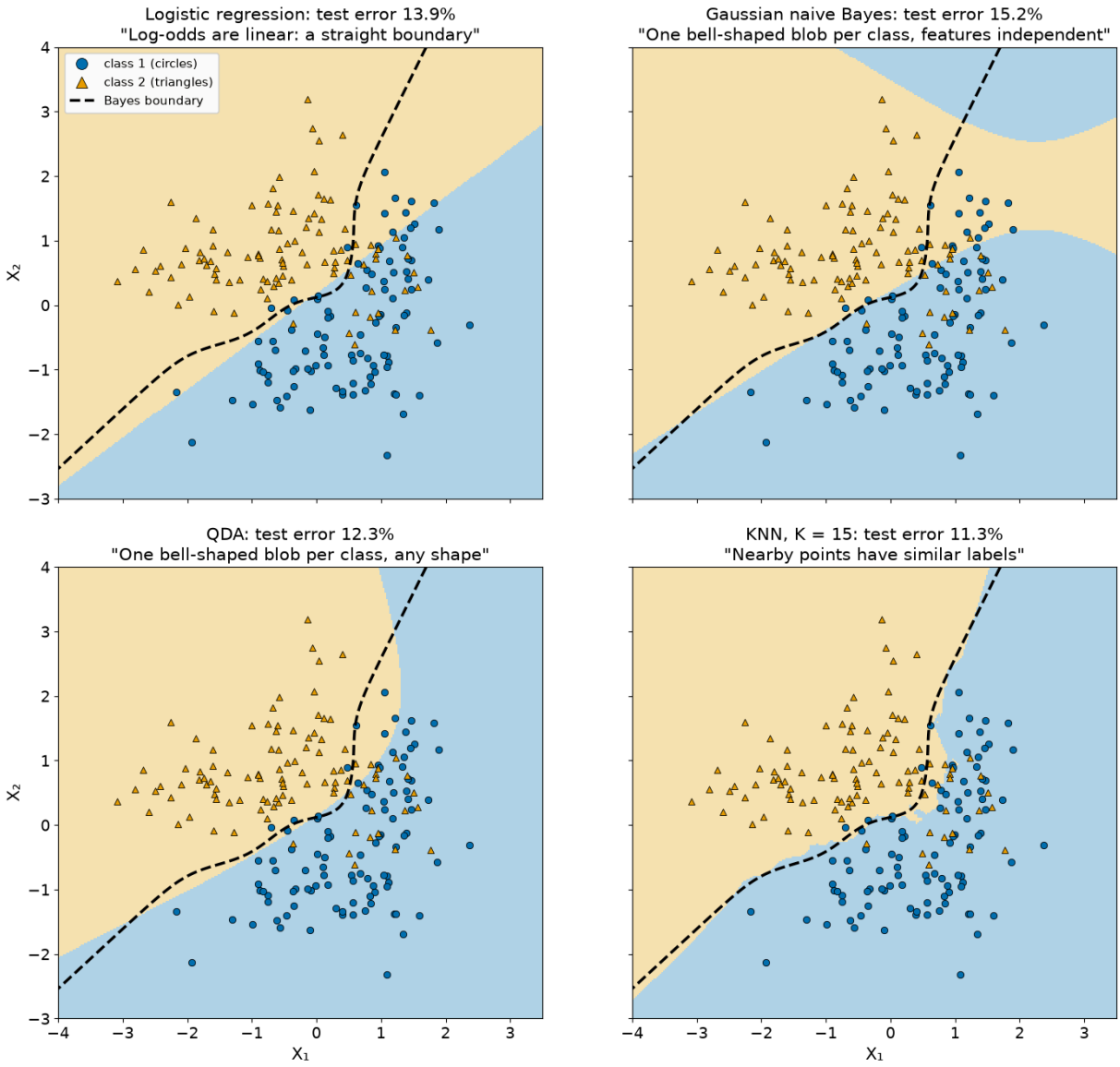
model	test error
Logistic regression	0.139
Gaussian naive Bayes	0.152
QDA	0.123
KNN, K = 15	0.113

Note: In the plots below, the features are continuous, so we model $f_j(x)$ in the Naive Bayes panel as bell-shaped blobs. This gives us a way to estimate the relevant probabilities.

Ask: Why is the decision boundary for logistic regression linear? See Demo 11.1: The probability surface in the Chapter 4 classification metrics notebook.

In [9]:
```
from ch_5_demos_helpers import plot_assumption_boundaries

plot_assumption_boundaries(sim, models);
```



Notice that the boundaries' shapes follow their assumptions. Logistic regression can only draw a straight line. Naive Bayes and QDA each draw a single smooth curve: they can bend, but only in one simple way. KNN follows the wiggles of the dashed curve, and it has the lowest test error.

Naive Bayes isn't wrong here because it's "naive." If each class really were generated by one round blob, its assumptions would match the truth exactly and it would find the Bayes boundary. It's wrong because each class is actually made of ten small bumps. A method's mistakes come from the mismatch between its assumptions and the world, not from the method alone.

Ask: Which of these boundaries could get close to the dashed curve if we gave it much more data? Which never could?

▼ Answer

Only KNN. A straight line stays a straight line however much data it sees, and one smooth blob per class stays one smooth blob, so neither can bend into the

dashed curve. That's the idea Demo E (Section 6b) calls bias.

6 · New data: test error

Every error rate in Demo B so far was measured on the training points. The **test error** is the error rate on new data from the same source: data the model never saw while it was being trained/fit. Because we are simulating data for these demonstrations, we can make as much new data as we like. The simulation also made 10,000 test points, 5,000 per class.

Ask: How do you think $K = 1$ will do on brand-new data from the same source?

▼ Answer

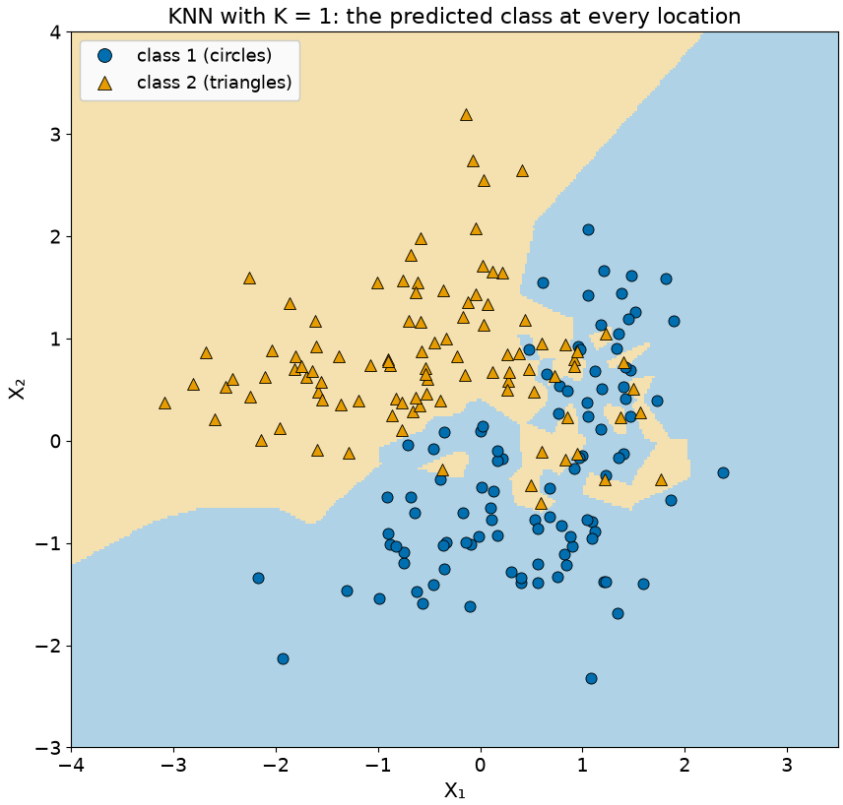
Much worse than its 0% training error, and worse than the Bayes error rate, since nothing beats the Bayes classifier on new data. The $K = 1$ boundary bends around every training point, including the ones on the "wrong" side by chance, and new points don't share those quirks.

Turn on **Show test error** in Demo B and slide K from one end to the other.

In [10]:
```
demo_b
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

K = 1, test error on



Training error: **0.0%**
Test error (10,000 new points): **16.8%**

Code worth reading. The loop fits KNN for each K on the slider and records both error rates in a small table. The model is fit to the training points only; the test points are used just for scoring.

In [11]:
```
X_test, y_test = sim.X_test, sim.y_test     # 10,000 new points from the sam

# It's worth reading the code below
rows = []
for K in [1, 3, 5, 7, 9, 11, 15, 21, 31, 41, 51, 71, 101, 151]:
    model = KNeighborsClassifier(n_neighbors=K)
    model.fit(X, y)
    rows.append({'K': K,
                 'training error': 1 - model.score(X, y),
                 'test error': 1 - model.score(X_test, y_test)})
errors = pd.DataFrame(rows).set_index('K')
errors.round(3)
```

Out[11]:

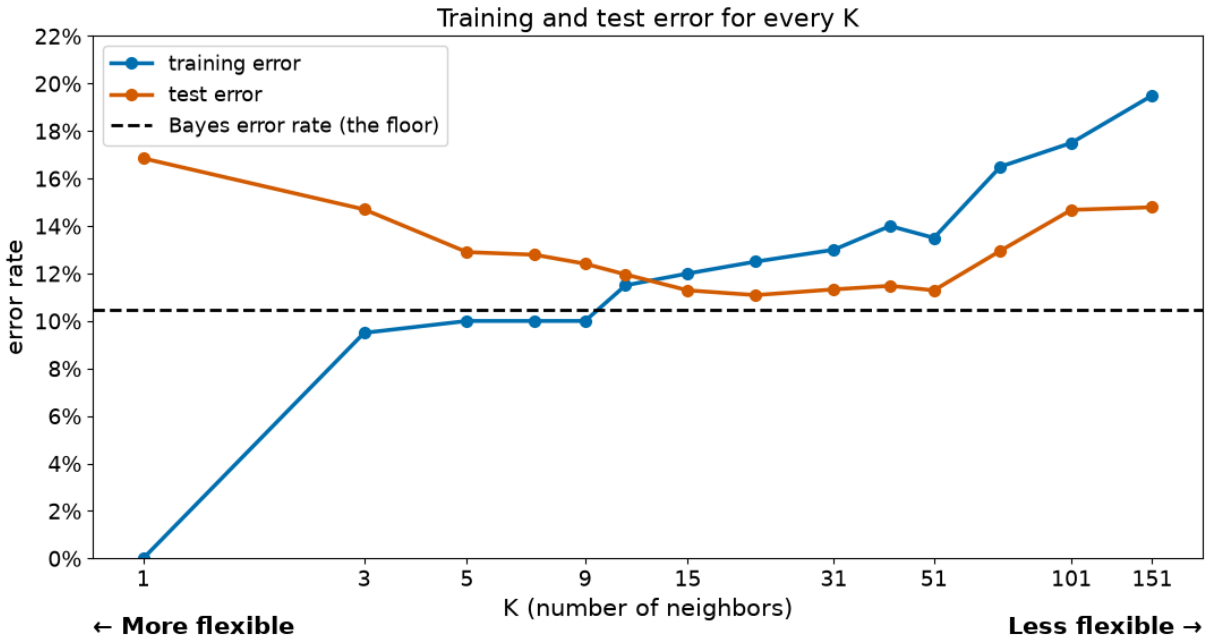
K	training error	test error
1	0.000	0.168
3	0.095	0.147
5	0.100	0.129
7	0.100	0.128
9	0.100	0.124
11	0.115	0.120
15	0.120	0.113
21	0.125	0.111
31	0.130	0.113
41	0.140	0.115
51	0.135	0.113
71	0.165	0.130
101	0.175	0.147
151	0.195	0.148

Demo D: Training versus test error over K

The plot shows both columns of the table against K, on a log scale, with the Bayes error rate as a dashed line. $K = 1$ is on the left, the same direction as Demo B's slider. ISL's Figure 2.17 plots the same idea with flexibility increasing to the right (it puts $1/K$ on the x axis), so its picture is the mirror image of this one.

In [12]:
```
from ch_5_demos_helpers import bayes_error_rate, plot_error_curves

plot_error_curves(errors, bayes_error_rate(sim));
```



Training error keeps falling as the model gets more flexible. Test error falls, then rises. At the flexible end, the model fits the noise in the training points. At the rigid (less flexible) end, it cannot follow the real shape of the boundary. Section 6b looks at why.

The best K is near the bottom of the test-error curve, close to the floor. But here we could make 10,000 brand-new points because we wrote the recipe. With real data, we can't. How do we find the bottom of that curve then?

Ask: Where would you choose K? What would you use to choose it if you only had the 200 training points?

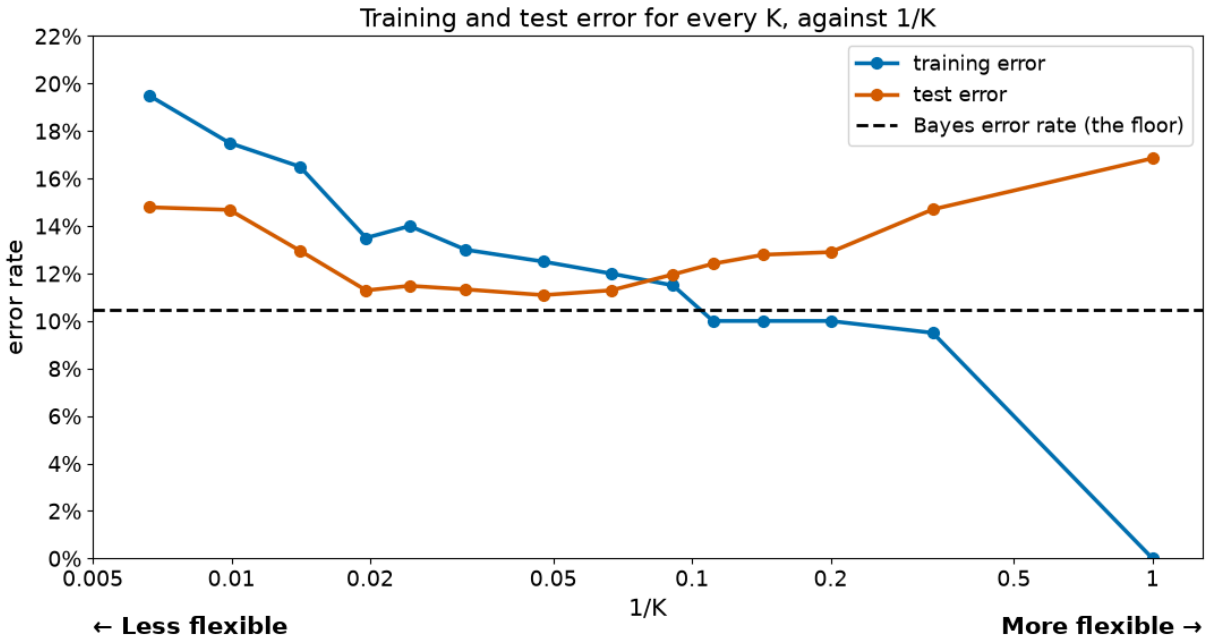
▼ Answer

Near the bottom of the test-error curve, around $K = 15$ to 50 here. Training error can't make that choice: it always points to $K = 1$. With only the 200 training points, we'd need a way to estimate test error from the training data itself, for example by setting some of the points aside to score the model on. That's the subject of Part 2.

The $1/K$ version

ISL's Figure 2.17, and most books, draw this plot so that flexibility increases to the right. For KNN, that means putting $1/K$ on the horizontal axis: $K = 151$ becomes about 0.007 on the left, and $K = 1$ becomes 1 on the right. It's the same table as above, mirrored.

In [13]:
```
plot_error_curves(errors, bayes_error_rate(sim), one_over_k=True);
```



6b · Bias and variance: redrawing the training data

Demo D showed test error falling and then rising as the model gets more flexible. Why? Since we wrote the recipe, we can do something impossible with real data: throw away the training set, draw a new one, refit, and watch how the boundary changes.

Demo E: Redraw the training data

- Generate 50 fresh training sets of 200 points each (plot displays up to 10 of these, the rest are used for the average boundary).
- Fit three methods to every one: KNN with $K = 1$, KNN with $K = 15$, and logistic regression.
- Each thin line is the boundary from one training set.

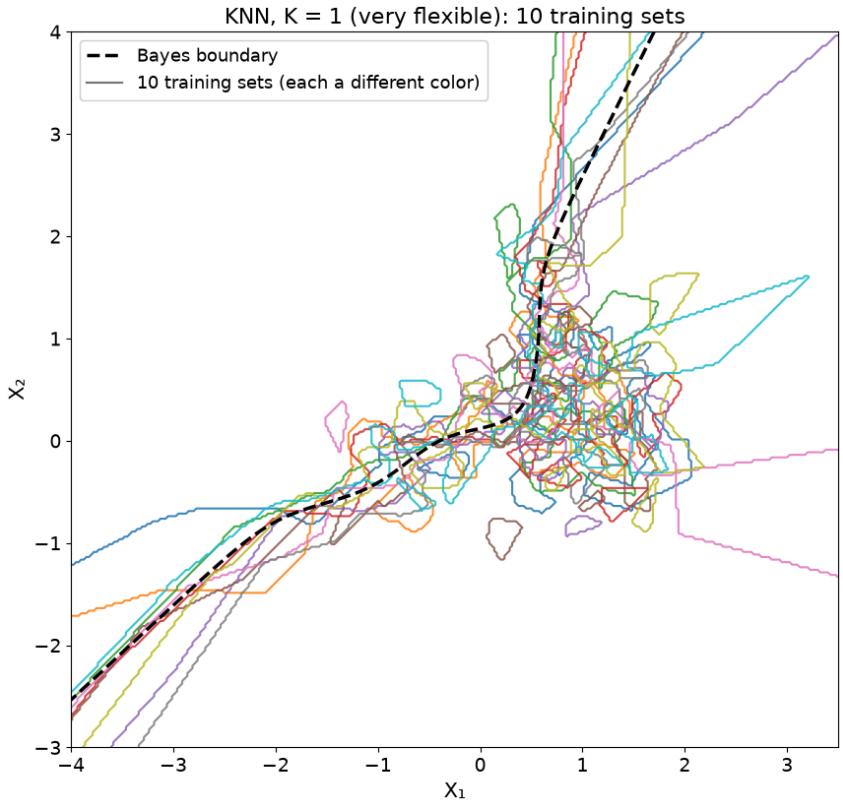
Start with **KNN, K = 1** and move **Training sets shown** up one step at a time.

In [14]:
```
from ch_5_demos_helpers import redraw_demo

demo_e = redraw_demo(sim, size_percent=80)
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

KNN, K = 1: 10 training sets



Variance: On average, **10%** of the plot changes class from one training set to another (averaged over all pairs of the 50 training sets).
Average test error: **15.8%** (averaged over the 50 training sets).

With highly flexible models, every redraw gives a very different boundary.

> **Variance** describes how much a method's boundary would change, *on average*, if we trained it on a different random sample of the same size from the same source.

The first readout estimates it for the selected method: the share of the plot that changes class between two training sets, averaged over all pairs of the 50 training sets.

Ask: The $K = 1$ boundaries are all over the place. Where do you think their average will land?

▼ Answer

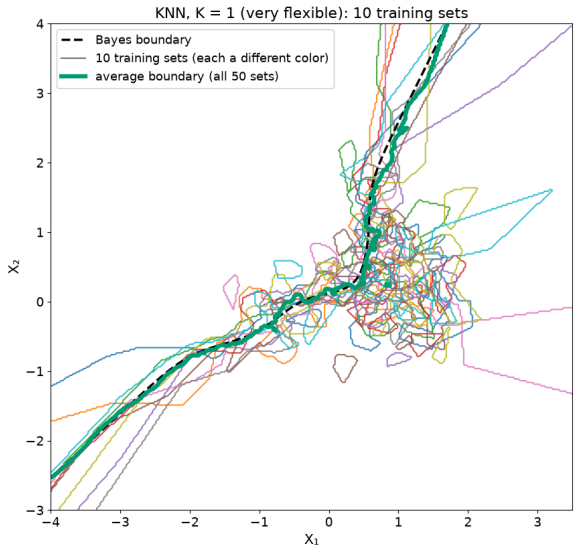
Close to the dashed Bayes boundary. Each fit is wild, but the errors point in different directions and mostly cancel out. That's low bias with high variance (bias is defined just below).

Turn on **Show average boundary** to check your answer. It uses all 50 training sets, however many are shown. Then switch to **Logistic regression (rigid)**. The demo is shown again below.

In [15]:
```
demo_e
```

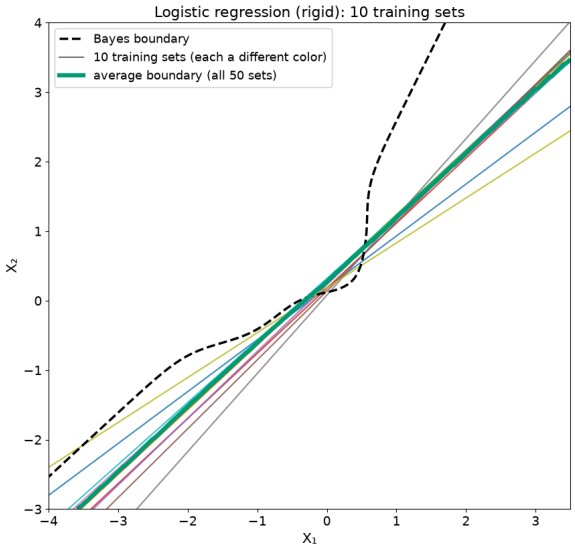
PDF version: the demo is shown at the settings below. Run the notebook to explore it.

KNN, K = 1: average boundary on



Variance: On average, **10%** of the plot changes class from one training set to another (averaged over all pairs of the 50 training sets).
Average test error: **15.8%** (averaged over the 50 training sets).

Logistic regression: average boundary on



Variance: On average, **4%** of the plot changes class from one training set to another (averaged over all pairs of the 50 training sets).
Average test error: **13.1%** (averaged over the 50 training sets).

Every logistic regression redraw gives almost the same line, and every one of them is wrong in the same way: it's a straight line, and the best possible boundary (dashed curve) isn't.

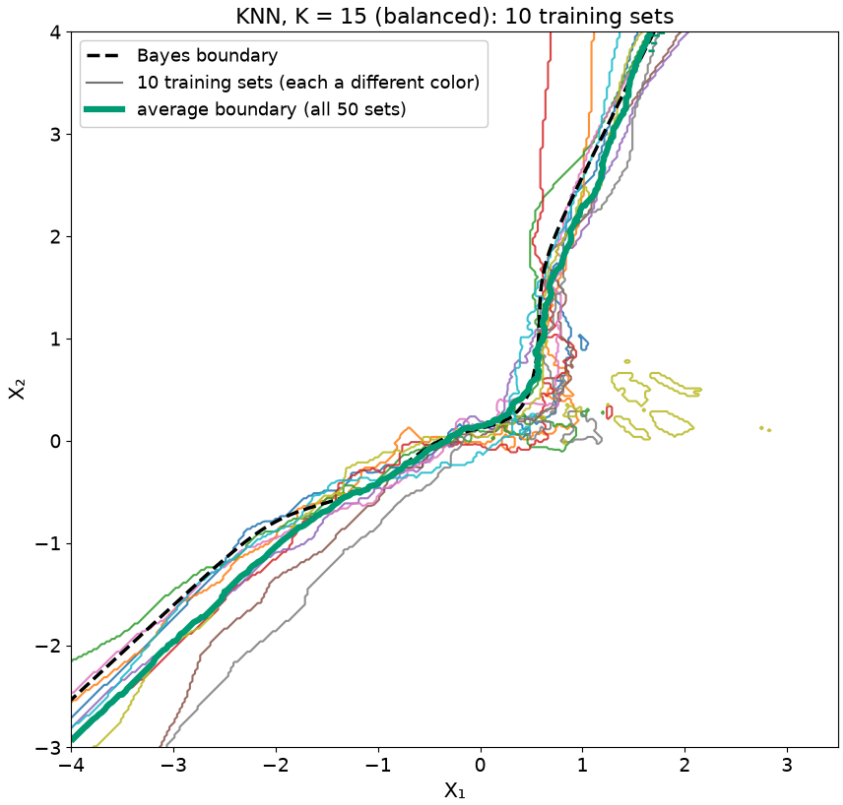
> **Bias** describes how far the method's *typical* boundary (its average over many training samples) is from the best possible boundary, the Bayes boundary.

Now switch to **KNN, K = 15 (balanced)**.

In [16]:
```
demo_e
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

KNN, K = 15: average boundary on



Variance: On average, **5%** of the plot changes class from one training set to another (averaged over all pairs of the 50 training sets).
Average test error: **11.2%** (averaged over the 50 training sets).

Some movement, and the average boundary follows the dashed curve closely. This is the balance.

Both of these describe a method and its settings, like KNN with $K = 15$, for data of a given kind and amount, not one fitted boundary. A single fitted boundary is just one curve; it's the method that would draw different curves from different samples. With more training data, or data from a different source, the same method can have a different bias and variance.

Too flexible means high variance; too rigid means high bias. Test error increases if bias is too high and if variance is too high, which is why it falls and then rises when plotted against model flexibility. The best model sits where the two are balanced: the bottom of the U. This is the **bias–variance tradeoff**. A method that's too flexible for the data (high variance) is **overfitting**, and one that's too rigid (high bias) is **underfitting**.

With real data we can't measure bias or variance directly, because we don't know the truth and only get one training set. But we can see their symptoms. High bias shows up as high error even on the training data. High variance shows up as a big gap between training error and error on new data.

ISL gives a formula that splits test error into bias and variance for regression (equation 2.7). For classification there's no equally clean formula, so we'll stick with the pictures.

Ask: In real life you get one training set, not 50. How could you tell whether your model is too flexible or too rigid?

▼ Answer
Look for the symptoms. If the error is high even on the training data, the method is probably too rigid (high bias). If the training error is low but the error on new data is much higher, it's probably too flexible (high variance). The second check needs an estimate of the error on new data from the one training set we have. Getting that estimate is what Part 2 is about.

7 · Part 1 summary

- KNN predicts by a vote of the K nearest training points, and the vote fraction estimates the probability of each class.
- Small K is flexible and large K is rigid. $K = 1$ always has 0% training error.
- When we know the recipe, equation 4.15 gives the true probability of each class everywhere, and the Bayes boundary is where the two are tied at 50%. On new data from the same source, on average, no classifier beats the Bayes classifier, yet its error rate isn't zero: that's irreducible error.
- A training error below the Bayes error rate means the model memorized noise.
- Training error keeps falling as flexibility grows; test error falls, then rises.
- Bias and variance describe a method and its settings, not one fitted boundary. On data like ours, KNN with $K = 1$ has high variance and logistic regression has high bias. Test error pays for both, so it is lowest where they are balanced: the bias–variance tradeoff.

The question Part 2 answers: with real data, we never know the recipe and can't make new data on demand. How do we estimate test error, and choose K, using only the data we have?

8 · Holding data back: the validation set approach

If we can't make new data, we can pretend some of ours is new. Split the 200 points in half at random, fit on one half, and measure the error on the other. This is the **validation set approach**. With only 100 training points in each half, K has to stay below 100, so these plots stop at $K = 71$.

Code worth reading. `train_test_split` shuffles the points and splits them: `test_size=0.5` puts half of them in the validation set, and `random_state=0` makes the split repeatable. The loop fits KNN on one half for each K, measures the error on the other half, and picks the K with the lowest validation error.

In [17]:
```
from sklearn.model_selection import train_test_split

K_values = [1, 3, 5, 7, 9, 11, 15, 21, 31, 41, 51, 71]

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.5, r
valid_errors = []
for K in K_values:
    model = KNeighborsClassifier(n_neighbors=K)
    model.fit(X_train, y_train)
    valid_errors.append(1 - model.score(X_valid, y_valid))
print('Chosen K:', K_values[np.argmin(valid_errors)])
```

```
Chosen K: 5
```

Demo F: The validation set approach, one split at a time

The plot starts with the true test-error curve from Demo D (thick black) and the Bayes error rate (dashed). Each click on **New random split** adds the validation-error curve for one more random split, and the table lists the K that split would choose. Split 1 is the split from the cell above.

Ask: If I split the data a different way, will I get the same best K?

▼ Answer

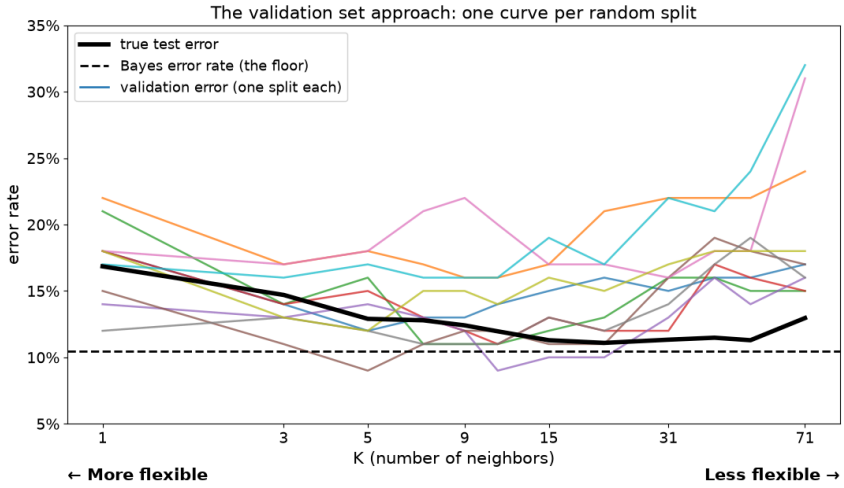
Usually not. Which points land in which half changes the curve, and with it the best K. Click a few times and watch the table.

In [18]:
```
from ch_5_demos_helpers import validation_set_demo

validation_set_demo(sim, errors.loc[K_values, 'test error'], size_percent=80
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

All 10 random splits



Average spread between the curves: **3.3** percentage points

split	chosen K
split 1	K = 5
split 2	K = 9
split 3	K = 7
split 4	K = 11
split 5	K = 11
split 6	K = 5
split 7	K = 31
split 8	K = 7
split 9	K = 5
split 10	K = 3

The validation set approach has two weaknesses:

1. The estimate, and the K it chooses, depend heavily on which points land in which half of the split (the training set or the test set). The curves and the table jump around from split to split.
2. Each model trains on only half the data, so it's typically a little worse than a model trained on all 200 points would be.

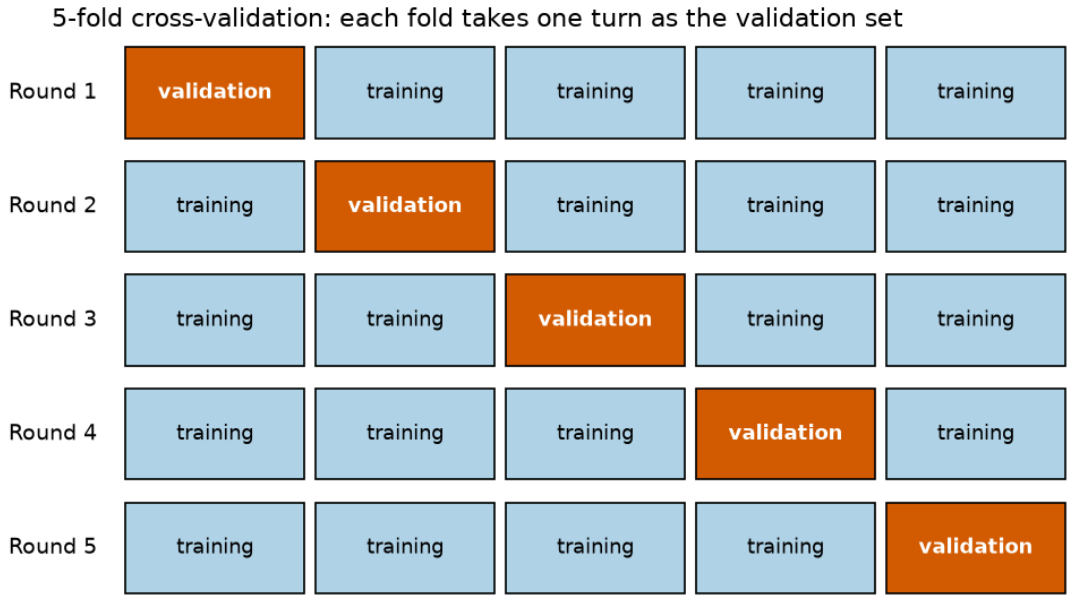
9 · k-fold cross-validation

Instead of splitting once, cut the data into k equal parts, called **folds**. Each fold takes one turn as the validation set while the model trains on the rest, and we average the k error rates. Every point is used for validation exactly once, and each model trains on most of the data. The diagram shows 5 folds; we'll use 10.

Demo G1: How k-fold cross-validation works

In [19]:
```
from ch_5_demos_helpers import plot_kfold_diagram

plot_kfold_diagram(n_folds=5);
```



Interpreting The Diagram: Within each round, *one* model is trained. All the blocks labeled as "training" are combined into a single training set per round.

Average the five validation errors (one per round/model) to get the cross-validation error.

Taken to the extreme, each fold is a single point, where $k = n$. That's called **leave-one-out cross-validation**, and it requires fitting the model n times (ISL discusses a surprising exception for linear regression that is far more efficient).

Ask: With 10-fold cross-validation on our 200 points, how many points does each model train on? How many times do we fit the model?

▼ Answer

With 5 folds, we assign 1/5 of the available data as validation data per round. In this example/question, we have 10 folds, so we assign 1/10th of the 200 points = 20 points to the validation set per round. So, we train each round's model on a different 200 - 20 = 180 points. These 180 points are made up of all 9 folds that aren't the validation fold. We fit the model 10 times (once per round), so we

evaluate 10 different versions of the model (we refer to these as separate models).

10 · Does cross-validation work?

Code worth reading. `KFold(n_splits=10, shuffle=True, random_state=0)` says how to cut the folds: shuffle the points in a repeatable way, then cut them into 10 folds. `cross_val_score` does the loop over the folds for us: it fits the model 10 times and returns the 10 scores.

In [20]:
```
from sklearn.model_selection import KFold, cross_val_score

folds = KFold(n_splits=10, shuffle=True, random_state=0)
cv_errors = []
for K in K_values:
    # cross_val_score reports accuracy by default, one score per fold
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=K), X, y, cv=f
    cv_errors.append(1 - scores.mean())
print('Chosen K:', K_values[np.argmin(cv_errors)])
```

```
Chosen K: 7
```

Demo G2: Does cross-validation work?

The same plot and controls as Demo F, with one difference: each click on **Add 10-Fold Run** adds a 10-fold cross-validation curve, using a different shuffle of the points into folds. Run 1 is the one from the cell above. **Show the validation-set curves for comparison** draws Demo F's ten curves faintly in the background.

Below:

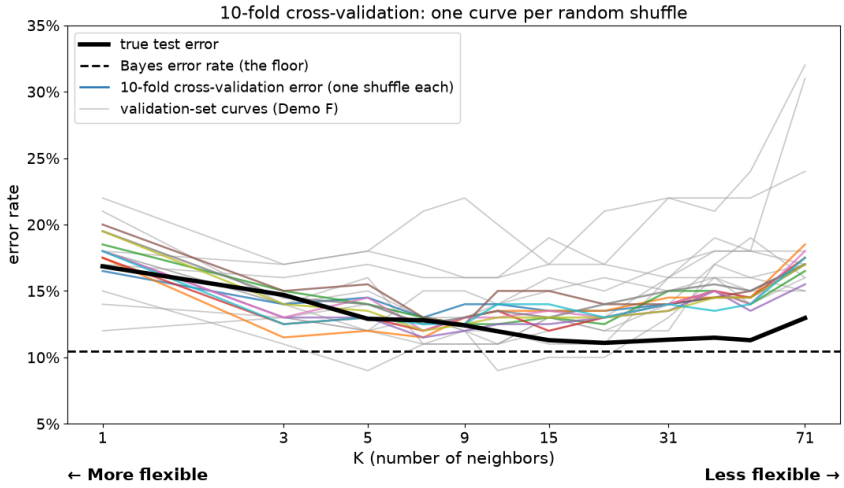
- 10-fold cross val error vs K
- validation set curves: train model on 1/2 the data and plot curve from measuring error on the hold-out (validation) set (the other half)

In [21]:
```
from ch_5_demos_helpers import cross_validation_demo

cross_validation_demo(sim, errors.loc[K_values, 'test error'], size_percent=
```

PDF version: the demo is shown at the settings below. Run the notebook to explore it.

All 10 runs, with the validation-set curves behind them



Average spread between the curves: **0.7** percentage points

run	chosen K
run 1	K = 7
run 2	K = 3
run 3	K = 9
run 4	K = 7
run 5	K = 7
run 6	K = 9
run 7	K = 7
run 8	K = 7
run 9	K = 7
run 10	K = 3

> The cross-validation curves agree with each other far more than the validation-set curves did. Each one averages 10 error rates, and every point gets used for validation.
>
> They also have the same shape as the true test-error curve: down, flat, then up. That's the curve we could only draw because we wrote the recipe, and cross-validation found its shape using nothing but the 200 training points.
>
> Notice they don't sit exactly on the true curve. Cross-validation is an **estimate** made from the same 200 points we trained on, so if those points happen to be a little easier or harder than typical, the estimate inherits that.

Ask: Why do the cross-validation curves agree with each other so much more than the validation-set curves?

▼ Answer
Each validation-set curve rests on one split: 100 validation points, chosen by chance. Each cross-validation curve averages 10 error rates and uses every one of the 200 points for validation, so no single lucky or unlucky split decides it.

11 · Using cross-validation

Choosing K. Near the bottom, the curve is nearly flat, so several values of K are about equally good, and which one wins can depend on the shuffle. That's fine. What cross-validation reliably does is steer you away from the bad ends: $K = 1$ and very large K.

Code worth reading. Choose K from the first cross-validation curve (shuffle 1), then refit on all 200 training points.

In [22]:
```
best_K = K_values[np.argmin(cv_errors)]
final_model = KNeighborsClassifier(n_neighbors=best_K).fit(X, y)     # all 2
print('Final model: KNN with K =', best_K)
```

```
Final model: KNN with K = 7
```

Refitting. Cross-validation is for *choosing*. Once you've chosen K, fit the final model on all your training data. Each cross-validation model saw only 180 points; the final one gets all 200.

Kaggle. Your project goes live this week on Kaggle. Its public leaderboard scores your predictions on one fixed part of the hidden test set, and the final ranking uses the rest. Tuning your model to climb the public leaderboard is the validation set approach: one split, used over and over. Teams that do it often drop when the final scores come out. Cross-validate on your training data instead.

One rule to remember. Any step that looks at the labels, like choosing features, has to happen inside cross-validation, or the estimate will look better than it really is.

Ask: On Kaggle, you could submit predictions 50 times and keep whichever scored best on the public leaderboard. What could go wrong?

▼ Answer
You'd be keeping whichever submission happened to fit the public part of the test set best: the validation set approach, used 50 times on the same split. Part

of that best score is luck that won't carry over, so the final ranking, on the rest of the hidden test set, will probably be worse.

Session summary

- Training error can't be trusted: the most flexible model scores perfectly on its training data and still does worse on new data.
- The Bayes error rate is the floor: on new/unseen data from the same source, no classifier does better on average.
- Bias and variance explain the U-shape of test error: too rigid means high bias, too flexible means high variance.
- Cross-validation estimates test error from the training data alone, so it can choose K when we can't make new data.

Learning Check Practice

The learning check covers the ideas below. Try each question on paper before you open its hint or answer. Each question links back to the demo or section it depends on.

Question 1: The vote

A query point x_0 has these 5 nearest neighbors: 2 circles (class 1) and 3 triangles (class 2). (See Demo A.)

(a) With $K = 5$, what does KNN predict for x_0?

(b) What is KNN's estimate of $\Pr(Y = 1 \mid X = x_0)$?

▼ Hint

The estimate is the fraction of the K nearest neighbors that are circles. Demo A's readout shows a similar fraction, for example "3/5 = 60% circle.", means the model predicts "circle" with 60% probability.

▼ Answer

(a) **Triangle.** Triangles have 3 of the 5 votes.

(b) $\Pr(Y = 1 \mid X = x_0) \approx \frac{2}{5} = 40\%$, means 40% probability of circle, so the model predicts x_0 is *not* a member of class circle. Two of the five neighbors are circles.

Question 2: Which K is more flexible?

Compare KNN with $K = 1$ and KNN with $K = 50$ on the same training data. (See Demo B.)

(a) Which one draws the more jagged boundary?

(b) Which one is more flexible?

▼ Hint

Slide K in Demo B from one end to the other. How many training points vote on each prediction at each end?

▼ Answer

(a) $K = 1$**.** Each prediction depends on a single nearby point, so the boundary bends around individual points.

(b) $K = 1$**.** Small K is the flexible end. With $K = 50$, 50 points vote on every prediction, so the boundary is smooth: that model is rigid.

Question 3: Training error at $K = 1$

Why does KNN with $K = 1$ always have 0% training error? (See Demo B.)

▼ Hint

When the model predicts the class of a training point, which training point is closest to it?

▼ Answer

Each training point is its own nearest neighbor, at distance 0. So when $K = 1$ predicts a training point, it looks up that point's own label, and it's always right. (This assumes no two training points sit at the same spot with different labels.)

Question 4: What the Bayes boundary is

Equation 4.15 gives the true probability of class 1:

$$\Pr(Y = 1 \mid X = x) = \frac{\pi_1 f_1(x)}{\pi_1 f_1(x) + \pi_2 f_2(x)}$$

(a) Why can we compute this for our simulated data? (See Section 3.)

(b) The Bayes boundary is the set of locations where $\Pr(Y = 1 \mid X = x)$ equals what number?

▼ Hint

Who chose π_1, π_2, $f_1(x)$, and $f_2(x)$ for the simulated data?

▼ Answer

(a) **We wrote the recipe.** We chose the priors π_1 and π_2 and the densities $f_1(x)$ and $f_2(x)$ that made the data, so we know every piece of the formula.

(b) **0.5.** On the boundary, the two classes are tied at 50/50. On one side class 1 is more likely, and on the other side class 2 is.

Question 5: The best choice at one location

At one location x, the true probabilities are 80% class 1 and 20% class 2. (See the argument in Section 3.)

(a) A classifier predicts class 2 at x. How often is it right there?

(b) What does the Bayes classifier predict at x, and how often is it right there?

(c) Even the Bayes classifier is sometimes wrong at x. What is this kind of error called, and why can't any classifier avoid it?

▼ Hint

Any classifier makes one prediction at x. Whichever class it predicts, it's right exactly as often as that class really occurs at x.

▼ Answer

(a) **20% of the time**, because class 2 occurs 20% of the time at x.

(b) **Class 1, right 80% of the time.** The Bayes classifier picks the more likely class, and nothing can be right more than 80% of the time at this x.

(c) **Irreducible error.** Both classes really occur at x, so any single prediction there is sometimes wrong. Averaging this error over the data gives the Bayes error rate.

Question 6: Beating the best possible classifier?

In Demo B, with the Bayes boundary on, KNN with $K = 1$ has 0% training error, below the Bayes error rate.

(a) Does this contradict the claim that no classifier beats the Bayes classifier? (See Section 4.)

(b) What does a training error below the Bayes error rate tell you about the model?

▼ Hint

The claim is about *new* data from the same source. Are the training points new to the model?

▼ Answer

(a) **No.** The claim is about new data. The training points aren't new: the model has already seen their labels.

(b) **The model memorized noise.** No rule can do that well on new data, so the extra accuracy comes from fitting chance quirks of these particular training points, including the ones on the "wrong" side of the Bayes boundary.

Question 7: Approximating the Bayes classifier

With real data, we never know the true priors π_j and densities $f_j(x)$, so every classifier approximates the Bayes classifier. Match each method to the assumption it uses to fill the gap. (See the table in Section 5.)

Methods: (a) logistic regression (b) naive Bayes (c) KNN

Assumptions: (i) the features are independent within each class (ii) nearby points have similar labels (iii) the log-odds are linear

▼ Hint

Two of these methods estimate $\Pr(Y = 1 \mid X = x)$ directly. One estimates the priors and densities, then uses Bayes' theorem.

▼ Answer

(a) Logistic regression: **(iii)**, the log-odds are linear.

(b) Naive Bayes: **(i)**, the features are independent within each class.

(c) KNN: **(ii)**, nearby points have similar labels.

Question 8: Training error versus test error

Look at the two error rate curves in Demo D.

(a) Which curve keeps falling as the model gets more flexible?

(b) If you chose K by the lowest training error, which K would you choose? How does that K do on the test-error curve?

▼ Hint

In Demo D, the flexible end is on the left. Find the lowest point of each curve.

▼ Answer

(a) **Training error.** It keeps falling as K gets smaller, all the way to 0% at $K = 1$.

(b) $K = 1$, the lowest training error. But on the test-error curve, $K = 1$ is near the worst: test error falls and then rises, and $K = 1$ sits at the high, flexible end. That's why training error should not be used to choose K.

Question 9: Bias or variance?

In Demo E, two methods are each trained on many fresh training sets from the same source.

- **Method A:** its boundaries look very different from one training set to the next, but their average lands close to the Bayes boundary.
- **Method B:** its boundaries are nearly the same every time, but their average is a straight line, far from the curved Bayes boundary.

(a) Which method has high variance on data like this? Which has high bias?

(b) In Demo E, which method behaves like Method A, and which like Method B?

▼ Hint

Variance is about how much a method's boundary would change, on average, from one training sample to another. Bias is about how far its typical (average) boundary is from the Bayes boundary.

▼ Answer

(a) **Method A has high variance** (and low bias). **Method B has high bias** (and low variance).

(b) Method A is like **KNN with** $K = 1$. Method B is like **logistic regression**, which always draws a straight line.

Question 10: Why the U-shape?

In Demo D, test error is high at both ends of the curve and lowest in the middle. Use bias and variance to explain:

(a) why test error is high at the flexible end (small K).

(b) why test error is high at the rigid end (large K).

▼ Hint

At each end, ask which of the two problems the method has on data like this: its boundary changes a lot from sample to sample, or its typical boundary is the wrong shape.

▼ Answer

(a) **High variance.** A very flexible method chases the particular points it happened to train on, so on average its boundary changes a lot from one training sample to another. That's called **overfitting**.

(b) **High bias.** A very rigid method is too inflexible to follow the real shape of the boundary, so its typical boundary is far from the Bayes boundary. That's **underfitting**.

Test error is impacted by both problems, so it's lowest in between, where they're balanced: the bias–variance tradeoff.

Question 11: Two classmates, two answers

Two classmates use the validation set approach on the same 200 training points. Each splits the points in half at random, fits KNN on one half for every K, and measures the error on the other half. One classmate's split chooses $K = 5$; the other's chooses $K = 31$. (See Demo F.)

Did one of them make a mistake? What weakness of the validation set approach does this show?

▼ Hint

Click **New random split** in Demo F a few times and watch the table of chosen K values.

▼ Answer

Probably neither made a mistake. The validation set approach's estimate, and the K it chooses, depend heavily on which points land in which half. A different random split can give a quite different curve and a different best K.

Question 12: How 5-fold cross-validation splits the data

Suppose you run 5-fold cross-validation on our 200 training points. (See Demo G1.)

(a) How many points are in each fold?

(b) How many points does each model train on?

(c) How many times is each point used for validation?

(d) Leave-one-out cross-validation uses one point per fold. How many times would it fit the model for each value of K?

▼ Hint

In the diagram, each row is one round. In each round, one fold is the validation set and the other folds are the training set.

▼ Answer
(a) $200/5 = $ **40 points**.

(b) The 4 folds that aren't validating: $4 \times 40 =$ **160 points**.

(c) **Exactly once.** Each fold takes one turn as the validation set.

(d) **200 times**, once per point, because $k = n = 200$.

Question 13: What cross-validation is good for

In Demo G2, the 10-fold cross-validation curves have the same shape as the true test-error curve (down, flat, then up), but they don't sit exactly on it.

(a) Why don't they sit exactly on the true curve?

(b) What can you still rely on cross-validation for?

▼ Hint
What data does cross-validation use to make its estimate? What would you need to get the true test error?

▼ Answer
(a) Cross-validation **estimates** test error using only our 200 training points, and those 200 points are just one random sample. By chance, a sample can have more points than usual in the region where circles and triangles overlap, where every classifier makes mistakes, or fewer. Cross-validation scores the model on these same points, so its estimate rises or falls with that luck. The true test error is measured (in this demo) on 10,000 new points, so this luck averages out.

(b) **Choosing K.** Its curves follow the true curve's shape, so it steers you away from the bad ends ($K = 1$ and very large K) toward the flat bottom, where several values of K are about equally good.

Question 14: After choosing K

10-fold cross-validation chooses $K = 7$. Which model should you use to make consequential predictions: one of the 10 models fit during cross-validation, or a new model? Explain. (See Section 11.)

▼ Hint
How many training points did each cross-validation model see? How many do you have?

▼ Answer

A new model: `KNeighborsClassifier(n_neighbors=7)` fit on all 200 training points. Cross-validation is for *choosing* K. Each cross-validation model saw only 180 points, and the final model should get all 200.